Neuromorphic Computing Final Project

Evolutionary Neural Architecture Search for Attention-Enabled Spiking Neural Networks

Chiara Benini, s1116239
Aalina Bilal, s1167114
Kristina Hristova, s1049501
Tim van de Pol, s1148777

Radboud University
January 25, 2026

Abstract

Spiking Neural Networks (SNNs) offer an energy-efficient and biologically inspired alternative to traditional neural networks, but their design and optimisation remain challenging due to non-differentiable spiking dynamics and temporal complexity. In this project, we explore the use of evolutionary Neural Architecture Search (NAS) to optimise attention-enabled SNNs. We integrate Spiking-Efficient Channel Attention (SECA) layers into a convolutional SNN and apply an evolutionary algorithm to optimise attention placement and key spiking parameters, including simulation time steps and membrane decay. Experiments on CIFAR-10 and CIFAR-100 demonstrate that the evolutionary approach identifies competitive architectures with performance comparable to a manually designed baseline, while underlining the important trade-offs between accuracy, spiking activity, and computational cost.

1 Introduction

Spiking Neural Networks (SNNs) have emerged as a promising alternative to traditional Artificial Neural Networks (ANNs) by offering a significantly improved energy efficiency and a closer biological resemblance. Unlike ANNs that propagate continuous real-valued activations through dense matrix operations at every layer, SNNs communicate exclusively via discrete asynchronous spikes and their neurons fire only when their membrane potential exceeds a certain threshold. These properties make SNNs particularly suitable for deployment on neuromorphic hardware and other low-power applications. Despite these advantages, designing and optimising SNN architecture remains a significant challenge. The binary and non-differentiable nature of spiking neurons prevents the direct application of standard gradient-based optimization techniques and often requires alternative approaches like surrogate gradients, ANN-to-SNN conversion, or derivative-free optimisation techniques. Moreover, SNNs require simultaneous optimization across spatial dimensions (layer topology, connectivity) and temporal dimensions (simulation time steps, membrane decay, firing thresholds), which creates a complex mixed discrete-continuous search space.

Attention mechanisms offer a promising direction for enhancing the representational capacity of SNNs by enabling dynamic feature recalibration. In traditional deep learning, attention has proven to be highly effective in focusing computations only on informative channels while suppressing redundant ones. This concept has been further adapted to the spiking domain though methods such as the Spiking-Efficient Channel Attention (SECA) layer represents one such adaptation, which allows SNNs to selectively emphasize important channels while preserving event-driven sparsity.

In this project, we investigate the integration of SECA layers into convolutional SNNs and explore whether evolutionary Neural Architecture Search (NAS) can be used to automatically optimise the attention-enabled SNNs. Specifically, we implement an SNN architecture inspired by Dan et al. (2024) and apply an evolutionary algorithm to optimise a set of attention parameters. With this approach, we aim to balance performance, efficiency, and model complexity, ultimately contributing to the growing field of neuromorphic architecture design by providing insights into

the collaborative potential of attention mechanisms and evolutionary optimisation in SNNs. Our research question, then, is defined as follows:

How does the utilization of an evolutionary algorithm for evolving the attention parameters of an attention-enabled SNN affect model performance?

2 Background

2.1 Spiking Neural Networks

SNNs represent a class of biologically inspired neural models that aim to mimic more closely the information processing mechanisms of the human brain compared to traditional ANNs. SNNs are often described as the third generation of neural network models, following threshold and sigmoidal networks, because they communicate using time-coded spikes rather than static activations (Maass, 1997). In these models, neurons emit discrete asynchronous spikes only when their membrane potential, obtained by integrating incoming weighted spikes, crosses a firing threshold, which provides a closer model to biological action potentials and synaptic dynamics (Maass, 1997; Tavanaei et al., 2019). This event-driven computation means that many units to remain inactive most of the time, so computation and memory are focused on only the active parts of the network. When implemented on neuromorphic hardware, this sparse activity pattern can lead to substantially improved energy efficiency compared to using conventional deep networks, making SNNs especially attractive for low-power and edge-AI deployments (Kundu et al., 2024; Roy et al., 2019). At the same time, SNNs offer increased biological plausibility in comparison to standard ANNs, as they explicitly incorporate temporal dynamics, spiking thresholds, and synaptic integration that are central to neuronal computation in the brain (Tavanaei et al., 2019; Neftci et al., 2019).

Despite their advantages, training SNNs presents many difficulties due to the non-differentiable nature of spike generation, and several training paradigms have been developed to address these challenges. Surrogate-gradient descent is currently the dominant approach for direct training of SNNs, replacing the binary spike function with a smooth surrogate derivative during backpropagation so that gradients can be propagated through spiking neurons (Neftci et al., 2019; Tavanaei et al., 2019). The Leaky Integrate-and-Fire (LIF) neuron, which integrates currents with membrane decay (β) and spikes upon threshold crossing, serves as its core computational unit (Gerstner et al., 2014). For digital simulation and gradient-based training, this is commonly discretized (Wu et al., 2018). The update rule for the membrane potential $V[t]$ at time step t becomes:

$$V[t] = \beta\, V[t-1] + (1-\beta) I[t] - S[t-1] V_{\text{th}},$$

where $\beta = \exp(-\Delta t/\tau_m)$ is the membrane decay factor, $I[t]$ is the weighted input at step t, and $S[t-1]$ is a binary spike indicator from the previous step. The term $-S[t-1]V_{\text{th}}$ models the reset after a spike.

In parallel, ANN-to-SNN conversion methods transfer learned representations from high performing ANNs to SNNs, usually via weight and threshold rescaling, which helps to narrow the accuracy gap while utilising efficient event-driven inference (Rueckauer et al., 2017).

2.2 Attention Mechanisms in SNNs

Building onto the event-driven computations of SNNs, another enhancement for the representational capacity of neural networks is the attention mechanism. Attention mechanisms enable models to dynamically select relevant features while suppressing irrelevant ones, which significantly boosts performance in vision and language-related tasks Vaswani et al. (2017). In computer vision, this is often achieved through channel attention modules, such as the Squeeze-and-Excitation (SE) block (Hu et al., 2018), which learn to recalibrate channel-wise feature responses by modelling interdependencies between channels. A more parameter-efficient evolution is the Efficient Channel Attention (ECA) module (Wang et al., 2020), which uses a fast one-dimensional convolution to

capture local cross-channel interactions. In SNNs, the integration of attention must account for binary spike data, which differs fundamentally from the continuous-valued activations in traditional ANNs. The need for such adaptation has led to the development of specialized spiking attention modules, such as the Spiking-Efficient Channel Attention (SECA) layer (Dan et al., 2024), which modifies the ECA approach to operate directly on spike trains. This preserves SNN event-driven sparsity, as attention computation activates only when informative spikes arrive, avoiding the constant matrix operations of ANN attention. In image classification, SECA enables SNNs to focus spiking activity on relevant channels, such as edges or textures, while suppressing background noise, which reduces unnecessary spikes and improves both accuracy and energy efficiency (Dan et al., 2024; Kundu et al., 2024).

2.3 Neural Architecture Search

NAS automates the design of neural network architectures by exploring a predefined search space of possible topologies, connections, and hyperparameters to optimize some performance objective, such as accuracy, latency, or energy consumption Elsken et al. (2019). NAS algorithms evaluate thousands of candidate architectures to discover configurations that outperform hand-crafted baselines (Bello et al., 2017). Major NAS paradigms include reinforcement learning (RL) controllers that sample architectures (Bello et al., 2017), gradient-based methods that relax discrete choices to continuous parameters via softmax or Gumbel-softmax (e.g., DARTS) (Liu et al., 2018), and evolutionary algorithms that iteratively evolve populations through selection, crossover, and mutation (Real et al., 2019).

For SNNs, NAS needs to be applied to a more complex search space that includes both convolutional layer configurations and spiking-specific parameters: simulation time steps T, membrane decay factors β, firing thresholds V_{th}, neuron models, synaptic sparsity patterns, and hardware-aware metrics like synaptic operations (SynOps) rather than FLOPs (Neftci et al., 2019). This discrete and high-dimensional search space makes gradient-based NAS challenging and necessitates the need for derivative-free approaches like evolutionary algorithms (Yan et al., 2024). Recent SNN-NAS advances emphasise multi-objective optimisation balancing accuracy against spike count, latency, and energy: evolutionary methods like MONAS-ESNN jointly optimize these objectives (Saghand and Lai-Yuen, 2025), while supernet-based approaches enable weight-sharing across candidate architectures to accelerate search (Svoboda and Adegbija, 2025).

2.4 Evolutionary Algorithms in NAS

Evolutionary Algorithms (EAs) are population-based derivative-free optimisation methods, which makes them particularly well-suited for NAS in complex search spaces (Real et al., 2019; Elsken et al., 2019). EAs maintain a population of candidate architectures by iteratively evolving better designs through three operations (Bäck et al., 1997):

- *Selection:* Randomly sampling k number of architectures, and selecting the best performing one as the parent one.
- *Crossover:* Combining parent architectures by inheriting layer configurations, filter sizes, etc.
- *Mutation:* Applying small random changes, like adding/removing layers, adjusting kernel sizes, etc.

The evaluation phase measures validation accuracy, latency, and any hardware-specific metrics like synaptic operations (SynOps) for SNNs, using weight-sharing or zero-shot proxies to reduce computational cost (Yan et al., 2024).

Unlike gradient-based NAS, which requires continuous relaxation, EAs excel in discrete mixed-type search spaces, typical for SNNs. This allows them to effectively optimise non-differentiable parameters like time steps T, membrane decay β, firing thresholds V_{th}, and the location of attention modules (Neftci et al., 2019). Regularized Evolution (RE) (Real et al., 2019) introduces tournament selection and age-based discarding to balance exploration and exploitation, achieving ImageNet SOTA with 4x less compute than RL-based NAS.

3 Related work

Several works adapt channel or spatio-temporal attention to operate directly on spike trains. Dan et al. (2024) introduce SA-SNN, a spiking attention network that integrates a Spiking-Efficient Channel Attention (SECA) block into convolutional SNNs, demonstrating improved accuracy on image classification benchmarks by modulating channel-wise spike activity. More recent architectures such as TCJA-SNN (temporal-channel joint attention) (Zhu et al., 2024) and STAA-SNN (spatio-temporal attention aggregation) (Zhang et al., 2025) achieve strong performance results but rely on manually-designed configurations and have increased computational overhead. These approaches show that attention can significantly enhance representational capacity in SNNs, but they generally rely on fixed, hand-designed attention configurations.

Applying NAS to SNNs introduces additional challenges due to temporal dynamics, discrete spiking behavior, and hardware-centric metrics such as SynOps instead of FLOPs (Neftci et al., 2019). Yan et al. (2024) propose an efficient SNN NAS framework based on a branchless spiking supernet and SynOps-aware multi-objective optimization, and report 8 to 67 times search speedups over earlier SNN-NAS methods while discovering architectures that improve both accuracy and energy efficiency. Survey work by Svoboda and Adegbija (2025) further categorizes SNN-NAS into global search, hardware co-exploration, and predictor-based search, highlighting the growing interest in automated SNN design.

Evolutionary algorithms have been successfully applied to NAS across a range of tasks including image classification, object detection, and speech recognition (Real et al., 2019; Ren et al., 2021). Regularized evolution has been shown to discover competitive image classifiers with substantially less compute than RL-based NAS (Real et al., 2019), while more recent work explores evolutionary multi-objective NAS balancing accuracy, model size, and latency (Saghand and Lai-Yuen, 2025; Pinos et al., 2022).

To the best of our knowledge, no prior work combines evolutionary NAS with SECA-style channel attention in SNNs for CIFAR-10/100. This project positions itself at the intersection of these lines of work by evolving SECA-equipped SNN architectures and empirically evaluating the trade-offs between accuracy, spiking sparsity, and computational efficiency.

4 Implementation

The proposed framework consists of an SNN model with an attention layer, based on the work of Dan et al. (2024). The concept, then, is based on a NAS method, in particular an evolutionary algorithm (EA). In our implementation, we apply an evolutionary (NAS) algorithm to optimize over a subset of model parameters, in an attempt to find an optimal structure for the SNN model. The code of the implementation can be found at `https://github.com/timvdp314/ru-nmc-project`.

4.1 SNN Model

The SNN model is based on work from Dan et al. (2024), with potentially additional attention layers added to it. The overall model structure is shown in Figure 1 with a detailed version of Block 1 and Block 2 shown in Figure 2 (with the assumption of width channel being 1.0), the only difference in Block 3 being the substitution of the Max-Pool layer with the Adaptive-Pool layer. In our implementation, the attention layers are evolved by the EA, meaning that the layers are either present or not, as determined by the EA. Note that, due to computational constraints, it was decided to shrink the model such that training could be completed in a reasonable amount of time.

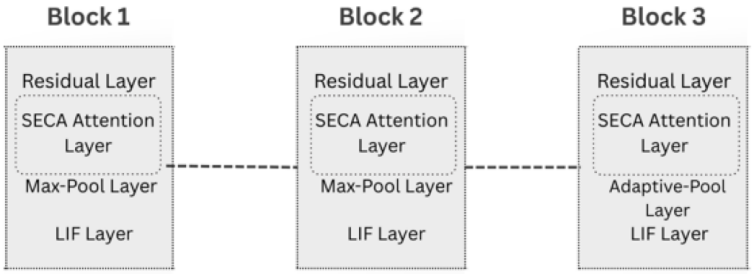

Figure 1: Attention SNN Architecture

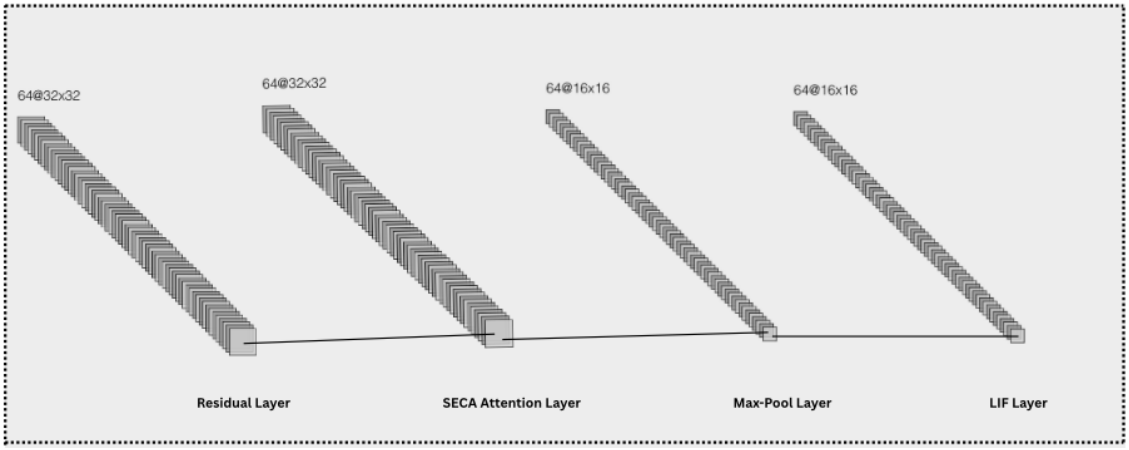

Figure 2: Attention SNN Single Block Detail

4.1.1 Attention

The attention layer is referred to as a Spiking-Efficient Channel Attention (SECA) layer, aimed at improving model accuracy by allowing the network to focus on important features and reduce the negative effects of redundant features (Dan et al., 2024). As explained by Dan et al. (2024), the attention block consists of:

- An **adaptive average pooling (2D)** layer: computes the average value over a channel and reshapes into 1x1 format;
- A **1-dimensional convolutional** layer: captures local cross-channel interactions;
- A **sigmoid** activation function: normalizes attention weights between 0 and 1;

4.1.2 Residual block

The residual blocks, consisting of a series of 2-dimensional convolutions and group normalizations, extract feature information from the spiking data. A diagram of a single residual block is shown in Figure 3.

4.1.3 Distributed Computing

In order to facilitate multi-GPU training, the PyTorch built-in mechanism *DistributedDataParallel* (The Linux Foundation, 2025) is utilized. Using the PyTorch backend, this allows for the training process to be parallelized, significantly decreasing the duration of the model training.

An important distinction between the original model (from Dan et al. (2024)) and our adapted model is how the number of spiking time steps is handled programmatically. Since our adapted

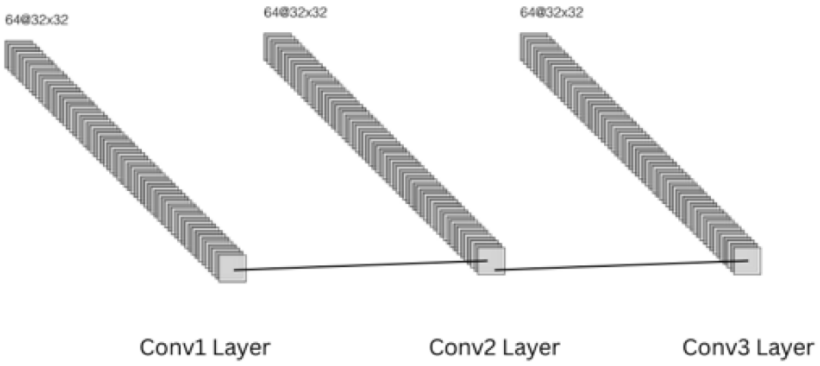

Figure 3: Residual Layer Architecture

model is run on a multi-GPU system, the training process can be distributed across multiple GPUs. This approach, however, is not compatible with the code of the original model, which runs a separate forward pass for n time steps. Since each GPU stores its own internal state (membrane potentials) separately from the other GPUs, this can result in network desynchronization. In practice, this simply results in the backward pass failing and a Python exception being thrown. Consequently, the code is adapted such that a full forward pass (for n time steps) is performed by each GPU, preventing synchronization issues.

4.2 Evolutionary Algorithm

The evolutionary algorithm is based on a simple evolutionary loop (see pseudocode in Figure 4).

```
BEGIN
  INITIALISE population with random candidate solutions;
  EVALUATE each candidate;
  REPEAT UNTIL ( TERMINATION CONDITION is satisfied ) DO
    1 SELECT parents;
    2 RECOMBINE pairs of parents;
    3 MUTATE the resulting offspring;
    4 EVALUATE new candidates;
    5 SELECT individuals for the next generation;
  OD
END
```

Figure 4: EA pseudocode (Eiben and Smith, 2015)

The parameters used by the EA are shown in Table 1. The candidates, in this case, are SNN models with a certain set of parameters (genotype). The value of a single parameter is refered to as a *gene*. The evaluation (fitness) of a candidate is determined by a short training process, in which the candidate is trained for a small number of epochs. The fitness is then simply determined by the prediction accuracy of the model. It was decided to train the candidates on a subset of the full dataset to reduce training time. While this somewhat reduces reliability of a model's prediction accuracy, it was deemed sufficiently reliable to obtain preliminary results on a model's training curve. The *termination condition*, in this case, is simply a set number of generations.
The EA optimizes over the model parameters as shown in Table 2. With regards to the attention layers, a tuple of three booleans indicates the presence (or absence) of the attention layers in Block 1, Block 2 and Block 3 (see Figure 1) respectively. Since the research focuses specifically on attention-enabled SNNs, it was chosen to enforce the presence of at least one attention layer, therefore omitting the "(0,0,0)" option for the *attention layers* parameter.

Table 1: EA evaluation parameters

Parameter	Value	Description
Evaluation epochs	4	Number of epochs for evaluation training
Evaluation train subset size	10000	Number of samples for evaluation training
Evaluation test subset size	2500	Number of samples for evaluation testing
Population size	6	Number of candidates per generation
Elite fraction	0.33	Fraction of candidates selected for the next generation
Mutation probability	0.1	Probability of random gene mutation

Table 2: EA search space

Parameter	Discrete values	Description
Attention layers	(1,0,0), (0,1,0), (0,0,1), (1,1,0), (1,0,1), (0,1,1), (1,1,1)	Topology of attention layers
Time steps	8, 10, 12, 14, 16, 18, 20	Number of steps performed in a single forward pass
β	0.85, 0.88, 0.90, 0.92, 0.95	Neuron membrane decay
Channel width	0.5, 1.0, 1.5, 2.0	Multiplier for residual block channel size

4.3 Training and evaluation

In order to be able to make a fair comparison, the following conditions have been evaluated:

- Baseline with one attention layer (based on model of Dan et al. (2024));
- An EA-generated model;

The baseline model uses the model parameters as shown in Table 3.

Table 3: EA baseline genotype

Parameter	Value
Attention layers	(1,0,0)
Time steps	12
β	0.90
Channel width	1.0

The performance metrics used to compare the baseline models and the EA model are as follows:

- Prediction accuracy (%);
- Average epoch duration (sec);
- Energy efficiency (average spiking rate);
- Sparsity (spiking correlation);

Since the original model by Dan et al. (2024) needed to be shrunk in order to comply with computational constraints, it was necessary to train the original model ourselves to obtain the desired metrics. Additionally, the authors of the original model did not include energy efficiency and sparsity in their performance metrics.

It was decided to use the CIFAR10 and CIFAR100 datasets as the training and testing datasets. Since simpler datasets such as MNIST and its variations (N-MNIST, FashionMNIST, ...) generally have quite high accuracy scores already (in the range of >90%), it was deemed more interesting to see whether our implementation could improve performance on a relatively more complex dataset.

Consequently, the training process for each dataset comprises three steps:

1. **Train baseline model**: The baseline model is trained (25 epochs) using a surrogate gradient (*atan*) and performance metrics are obtained;
2. **NAS using EA**: NAS is performed using the EA described previously. In order to get preliminary results of a model's performance, the EA performs a small amount of epochs to obtain performance metrics. In the end, the best-scoring model parameters are saved for further training;
3. **Train EA-generated model**: The best-scoring SNN model is trained (25 epochs) using a surrogate gradient (*atan*) and performance metrics are obtained.

Finally, the EA-generated model is compared with the baseline models to determine the effect of the EA. It was decided to use a relatively small amount of epochs for the final training session, mainly due to time constraints.

5 Results

The average loss and accuracy per generation of the evolutionary process for CIFAR10 and CIFAR100 are shown in Figure 5 and Figure 6 respectively.

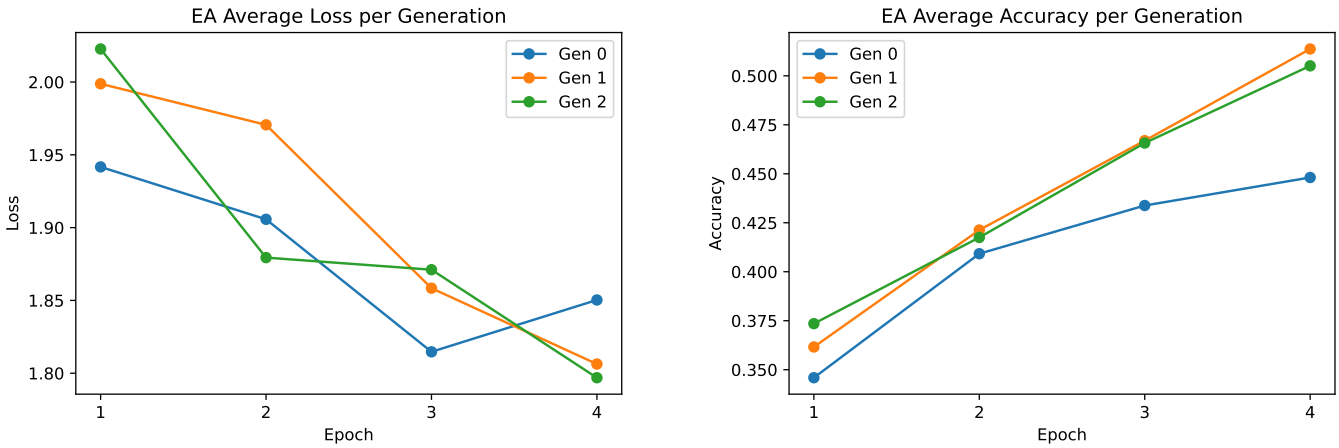

Figure 5: EA average accuracy and loss per generation (CIFAR10)

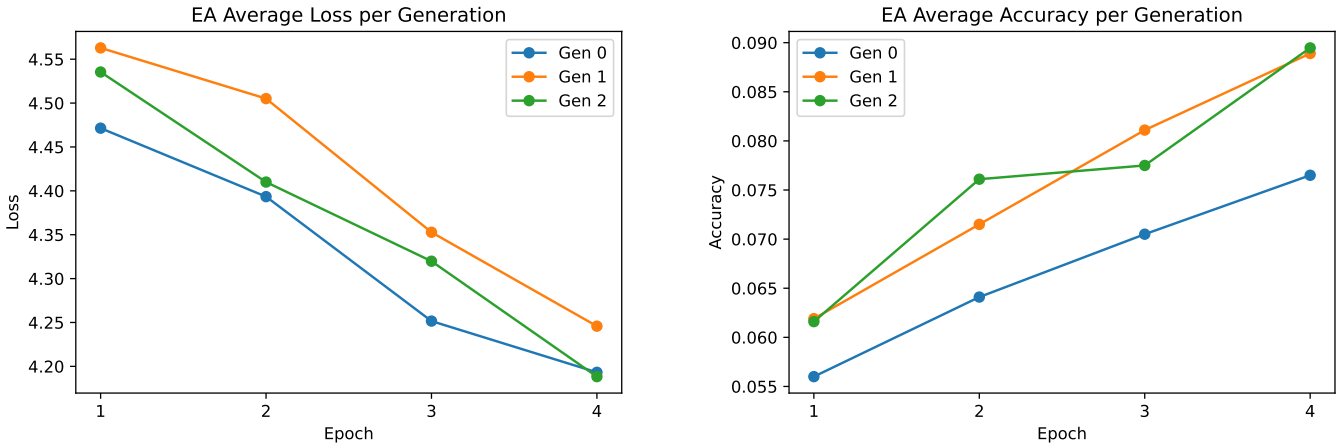

Figure 6: EA average accuracy and loss per generation (CIFAR100)

The genotype of the models with the highest fitness (accuracy) after the evolutionary process are shown in Table 4.

Table 4: EA best genotype

Parameter	**CIFAR10**	**CIFAR100**
Attention layers	(1,0,0)	(1,1,0)
Time steps	20	18
β	0.92	0.9
Channel width	1.0	0.5

The loss history and accuracy record of the baseline and evolved models for CIFAR10 and CIFAR100 are shown in Figure 7 and Figure 8 respectively.

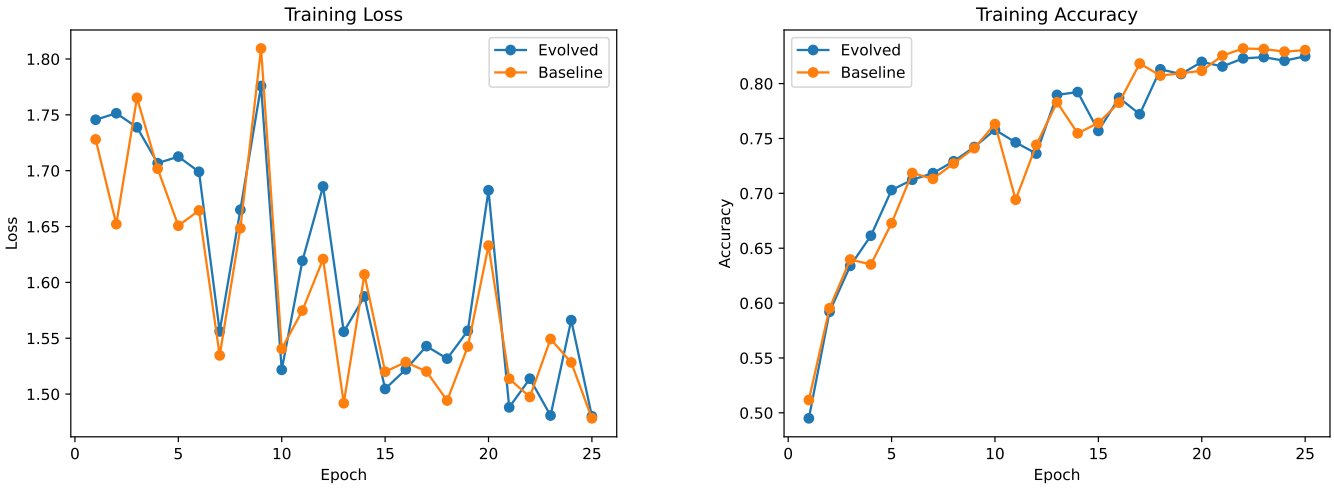

Figure 7: Training loss and accuracy (CIFAR10)

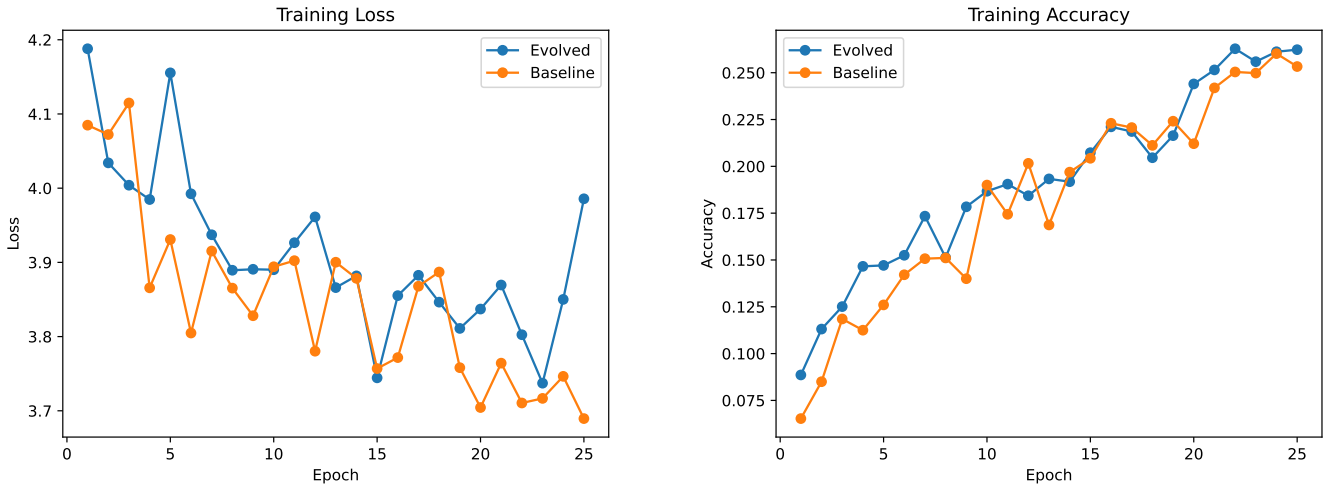

Figure 8: Training loss and accuracy (CIFAR100)

Some additional metrics are shown in Figure 9.

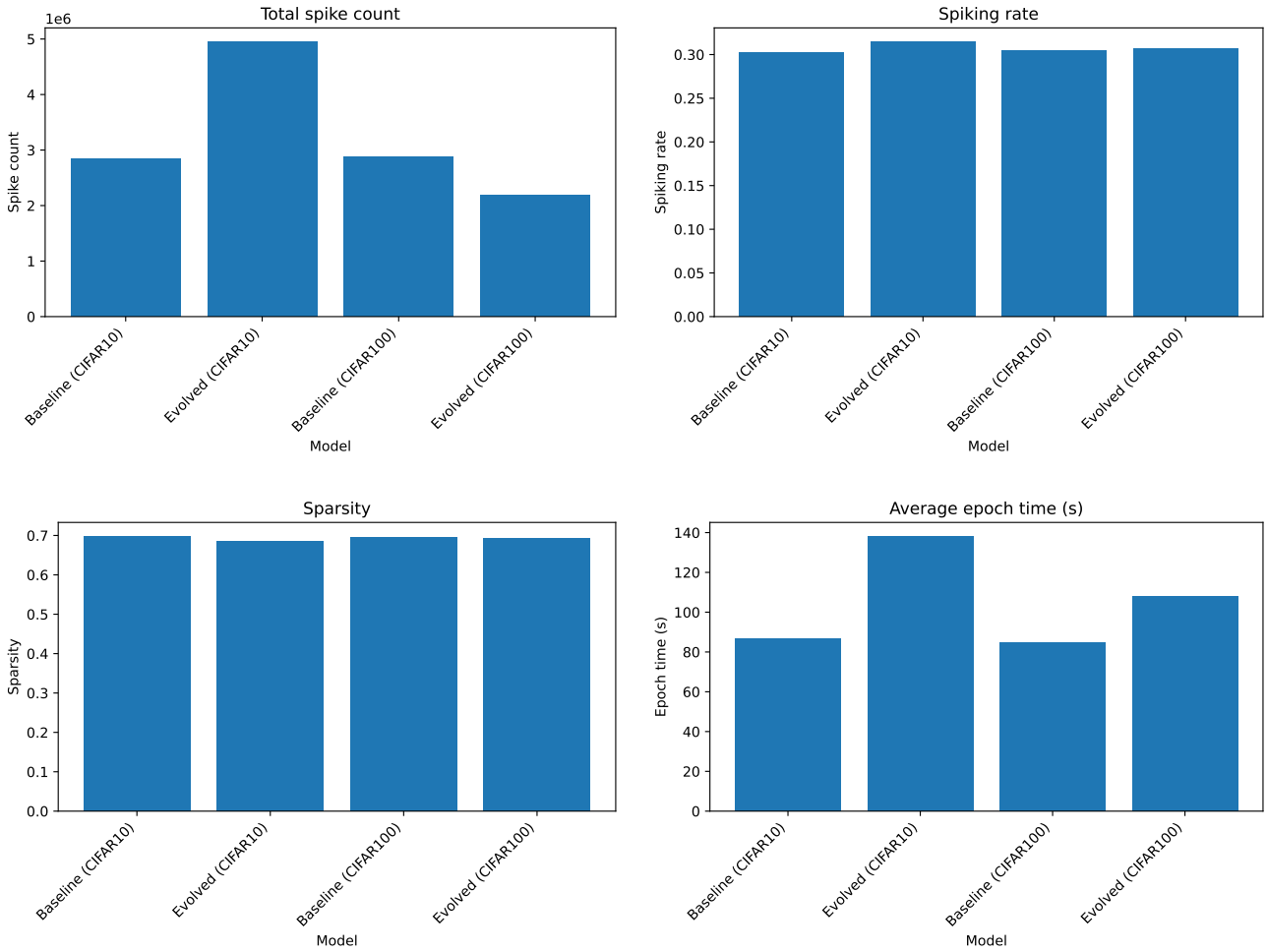

Figure 9: Additional performance metrics

6 Discussion

This section discusses the results comparatively, provides suggestions for future research and describes challenges that were encountered during the project.

6.1 Comparison

Looking at the results, it can be observed that, despite the relatively small search space, the EA successfully finds models for both CIFAR10 and CIFAR100 with the highest preliminary prediction accuracy and the lowest loss. Interestingly, the most fit candidate for the CIFAR10 dataset has the same attention topology as the baseline model. On the other hand, the EA found a different attention topology for the CIFAR100 dataset model.

Subsequently, looking at the full training results, there is no conclusive evidence that either the evolved or baseline model performs significantly better or worse, as the models have very similar loss/accuracy graphs. While small differences do exist, it is hard to claim that this could not be a result of noise or random chance. Then again, the amount of epochs used (25) to train the models is relatively small for SNNs and the datasets that were used. Additionally, the search space for EA was relatively small, which by definition makes it less likely for drastic changes to occur in a model's performance.

Furthermore, it is worth noting that both EA-generated models have increased time step values, which generally results in decreased fluctuations in the accuracy graph, but also increases

the epoch time significantly. The increase in total spikes for the evolved CIFAR10 model can mostly be explained by the increase in the β-parameter, making it more likely for spikes to occur. It was hypothesized that an additional attention layer for the evolved CIFAR100 model would have a greater impact on sparsity, but this turned out not to be the case. It could be the case, however, that this effect would only become significant through a more extensive training process (i.e., more training epochs). Moreover, given the smaller channel width (0.5) for the evolved CIFAR100 model, a significant reduction in total spikes can be observed, while retaining comparable accuracy to the baseline model.

6.2 Future research

There are quite some options available for improvement of our methodology. First and foremost, ideally one would evolve attention-specific parameters with the EA, such as convolutional parameters (kernel size, padding, stride, ...) or activation function type (*sigmoid*, *tanh*, ...), as well as the number of attention layers. In our work, only the latter was implemented, mainly due to time constraints. Therefore, we highly recommend expanding the search space of the EA in this manner.

Secondly, a somewhat simplified version of the original model by Dan et al. (2024) was utilized, mainly due to computational constraints. Ideally, one would use the original model without modifications instead, as this allows for a more reasonable comparison between the original and the adapted model with the EA.

Additionally, while the current implementation collects metrics about prediction accuracy, learning efficiency, model size and energy efficiency, not all of these are optimized over by the EA. In contrast, only prediction accuracy is used, which is reasonable, but ideally the EA would also be able to optimize the other three parameters. Again, this increases the computational complexity of the EA, as multi-objective optimization generally takes longer to converge, therefore we decided against implementing it. For future research, we suggest to implement a fitness function which takes a discounted value of the other three parameters. After all, prediction accuracy is most likely the top priority, therefore the other parameters should be weighed less in the fitness function.

Overall, the experiments conducted were relatively small and could easily be upscaled. While our results showed little variation in loss/accuracy for the evolved and baseline models, we believe that a more comprehensive study, for instance by expanding the search space of EA, the amount of epochs for full training and using more different datasets, would be a worthwhile endeavour. Most importantly, we believe that not only model accuracy should be taken into account, but also other performance metrics such as sparsity and training time.

6.3 Challenges

While the general concept of an evolving attention mechanism was present from the start, it was found to be quite difficult to find a source model which was relatively digestible in terms of theoretical complexity and model size. Some alternatives considered, for example, were TCJA-SNN (Zhu et al., 2024) and STAA-SNN (Zhang et al., 2025), however these were deemed too complex.

Furthermore, ideally one would get a multi-GPU setup functional to be able to more efficiently evaluate our implementation. It turned out to be quite challenging and time consuming to integrate the EA with the SNN model training, with multi-GPU functionality on top of that. While a functional multi-GPU script was made, the GPU utilization of the HPC cluster, provided by Radboud University, was frequently high, which resulted in many GPUs already being occupied. Therefore, we unfortunately were only able to use the multi-GPU setup a couple of times throughout the project, and instead had to resort to a single-GPU setup. Most of the limitations of the project, such as training time and model complexity, were either a result of time constraints or computational constraints.

7 Conclusion

In this project, we investigated the potential of using evolutionary NAS to optimise the attention parameters of SNNs. By integrating an evolutionary algorithm into the design of an SNN and evaluating its performance on CIFAR-10 and CIFAR-100, we aimed to explore whether automated optimisation can improve the SNNs' performance. Our results show that the evolutionary algorithm is capable of identifying good SNN configurations within a constrained search space, particularly by adapting the simulation time steps and the membrane decay. While the evolved models did not significantly outperform the baseline models in terms of accuracy, they performed comparatively in terms of learning behaviour. The experiment also revealed the meaningful trade-offs between computational cost, spiking activity, and training time. Our findings suggest that, even under strict computational constrains, NAS can prove to be beneficial for applications with a complex discrete-continuous design space such as that for SNNs.

This project also reflects the main learning objectives of the course. The design and exploration of spiking models allowed us to develop a deeper understanding of the core neuromorphic concepts across neuroscience, machine learning, and hardware-aware computation, which allowed us to compare spiking and conventional neural architectures in terms of efficiency and computational specificities. The hands-on experience of implementing and training SNNs, integrating attention mechanisms, and applying evolutionary algorithms forced us to reflect and consider the technical consequences of the design choices we made. Decisions such as restricting the search space, prioritizing accuracy over energy-aware optimization, and simplifying existing architectures due to computational limitations all mirror ongoing challenges in the field of neuromorphic computing. Finally, the collaborative part of this project reinforced the importance of clear communication in presenting and motivating our ideas and the balance needed for coherent integration of diverse contributions.

To conclude, although the performance improvements we observed were minimal, this project demonstrates the feasibility of combining attention mechanisms with evolutionary NAS for SNNs. More importantly, it provides practical insights into the challenges of automated SNN design and presents an opportunity for future work to focus on larger search spaces, multi-objective optimisation, and hardware-aware neuromorphic architectures.

References

Bäck, T., Fogel, D. B., and Michalewicz, Z. (1997). Handbook of evolutionary computation. *Release*, 97(1):B1.

Bello, I., Zoph, B., Vasudevan, V., and Le, Q. V. (2017). Neural optimizer search with reinforcement learning. In *International Conference on Machine Learning*, pages 459–468. PMLR.

Dan, Y., Wang, Z., Li, H., and Wei, J. (2024). Sa-snn: spiking attention neural network for image classification. *PeerJ Computer Science*, 10:e2549.

Eiben, A. E. and Smith, J. E. (2015). What is an evolutionary algorithm? In *Introduction to evolutionary computing*, pages 15–35. Springer.

Elsken, T., Metzen, J. H., and Hutter, F. (2019). Neural architecture search: A survey. *Journal of Machine Learning Research*, 20(55):1–21.

Gerstner, W., Kistler, W. M., Naud, R., and Paninski, L. (2014). *Neuronal dynamics: From single neurons to networks and models of cognition*. Cambridge University Press.

Hu, J., Shen, L., and Sun, G. (2018). Squeeze-and-excitation networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 7132–7141.

Kundu, S., Zhu, R.-J., Jaiswal, A., and Beerel, P. A. (2024). Recent advances in scalable energy-efficient and trustworthy spiking neural networks: from algorithms to technology. In *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 13256–13260. IEEE.

Liu, H., Simonyan, K., and Yang, Y. (2018). Darts: Differentiable architecture search. *arXiv preprint arXiv:1806.09055*.

Maass, W. (1997). Networks of spiking neurons: the third generation of neural network models. *Neural networks*, 10(9):1659–1671.

Neftci, E. O., Mostafa, H., and Zenke, F. (2019). Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks. *IEEE Signal Processing Magazine*, 36(6):51–63.

Pinos, M., Mrazek, V., and Sekanina, L. (2022). Evolutionary approximation and neural architecture search. *Genetic Programming and Evolvable Machines*, 23(3):351–374.

Real, E., Aggarwal, A., Huang, Y., and Le, Q. V. (2019). Regularized evolution for image classifier architecture search. In *Proceedings of the aaai conference on artificial intelligence*, volume 33, pages 4780–4789.

Ren, P., Xiao, Y., Chang, X., Huang, P.-Y., Li, Z., Chen, X., and Wang, X. (2021). A comprehensive survey of neural architecture search: Challenges and solutions. *ACM Computing Surveys (CSUR)*, 54(4):1–34.

Roy, K., Jaiswal, A., and Panda, P. (2019). Towards spike-based machine intelligence with neuromorphic computing. *Nature*, 575(7784):607–617.

Rueckauer, B., Lungu, I.-A., Hu, Y., Pfeiffer, M., and Liu, S.-C. (2017). Conversion of continuous-valued deep networks to efficient event-driven networks for image classification. *Frontiers in neuroscience*, 11:682.

Saghand, E. G. and Lai-Yuen, S. K. (2025). Monas-esnn: Multi-objective neural architecture search for efficient spiking neural networks. In *2025 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 2963–2972. IEEE.

Svoboda, K. and Adegbija, T. (2025). Spiking neural network architecture search: A survey. *arXiv preprint arXiv:2510.14235*.

Tavanaei, A., Ghodrati, M., Kheradpisheh, S. R., Masquelier, T., and Maida, A. (2019). Deep learning in spiking neural networks. *Neural Networks*, 111:47–63.

The Linux Foundation (2025). Distributeddataparallel. PyTorch v2.9.1.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, 30.

Wang, Q., Wu, B., Zhu, P., Li, P., Zuo, W., and Hu, Q. (2020). Eca-net: Efficient channel attention for deep convolutional neural networks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11534–11542.

Wu, Y., Deng, L., Li, G., Zhu, J., and Shi, L. (2018). Spatio-temporal backpropagation for training high-performance spiking neural networks. *Frontiers in neuroscience*, 12:331.

Yan, J., Liu, Q., Zhang, M., Feng, L., Ma, D., Li, H., and Pan, G. (2024). Efficient spiking neural network design via neural architecture search. *Neural Networks*, 173:106172.

Zhang, T., Yu, K., Zhong, X., Wang, H., Xu, Q., and Zhang, Q. (2025). Staa-snn: Spatial-temporal attention aggregator for spiking neural networks. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 13959–13969.

Zhu, R.-J., Zhang, M., Zhao, Q., Deng, H., Duan, Y., and Deng, L.-J. (2024). Tcja-snn: Temporal-channel joint attention for spiking neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 36(3):5112–5125.